

Python Cheat Sheet for the AI Era

AI writes the first draft.
You still own what ships.

- Requirements
- Reading code
- Logic
- Errors
- Tests
- Secrets
- Dependencies
- Maintainability

Sagar Rathkanthiwar
Data & AI Professional

12 slides • save for later →

It runs. That doesn't make it right.

90% use it • 30% barely trust it

DORA surveyed ~5,000 practitioners: over 80% report AI made them more productive, yet 30% report little or no trust in the code it returns.

45% of samples had a security flaw

Veracode tested 100+ models. Python failed 38–45% of security tasks — and security stayed flat even as the models got better at correctness.

1 in 5 suggested packages didn't exist

Across 576,000 generated samples, 19.7% of imported packages were hallucinated — and 43% of those fake names reappear on repeat prompts.

The scarce skill moved from **writing** Python to **judging** Python. Everything after this slide is the judging part.

Split the work into three lanes

ASK AI

First drafts, boilerplate, docstrings, test scaffolding, alternate approaches, and plain-English walkthroughs of code you inherited.

VERIFY YOURSELF

Requirements, data shapes and types, business rules, error paths, the security surface, and whether the output actually reconciles.

CATCH BEFORE SHIPPING

Bare `except`, hard-coded keys, `eval`, invented or unpinned packages, $O(n^2)$ loops, mutable defaults, and code with no tests.

AI is fast at Python. You are accountable for the business rule, the blast radius, and the person who maintains this next.

A vague prompt is a vague spec

Weak prompt "Write a Python script to clean my sales data."

```
Input:  sales.csv, ~2M rows
        order_id (str), order_ts (ISO 8601, may be blank),
        amount (str, may contain "$" and ","), region (str)
Output: DataFrame, one row per order_id, amount as float
Rules:  drop blank order_ts; uppercase region;
        duplicate order_id -> keep the latest order_ts
Limits: pandas 2.x, runs in under 60s, no network calls

Then list your assumptions and the edge cases you skipped.
```

Catch: code with no stated assumptions still has assumptions — you just inherit them silently. Make "list what you assumed" the last line of every prompt.

Know what each line hands back

Most AI bugs are **shape and type** bugs. The model guessed what your data looks like; it has never seen it.

```
row = {"id": 7, "tags": ["a", "b"], "amount": None}

row["email"]           # KeyError -> crash in production
row.get("email", "")   # safe default
row["tags"][0]         # IndexError if the list is empty

# Nested access AI assumes is always populated
row["user"]["email"]   # can KeyError at either level
(row.get("user") or {}).get("email")

# Every one of these is falsy: 0 0.0 "" [] {} None
if row["amount"]:      # silently skips a real 0.0
if row["amount"] is not None: # what you actually meant
```

Ask AI: "Walk through this line by line and tell me the type of every name." Then confirm it against real data with `type()`, `len()` and `df.dtypes`.

Shorter isn't the goal. Clear and cheap is.

```
# Common first draft
result = []
for r in rows:
    if r["region"] == "US":
        result.append(r["amount"] * 1.08)

# Same thing - fine while it stays this simple
result = [r["amount"] * 1.08 for r in rows if r["region"] == "US"]

# Hidden O(n^2): a list scan inside a loop over n rows
flagged = [r for r in rows if r["id"] in blocked_list] # slow
flagged = [r for r in rows if r["id"] in blocked_set] # O(1) lookups
```

- **Nested or triple-conditional comprehensions** are worse than the loop. Write the loop.
- **Row-by-row loops over a DataFrame** (`iterrows`) are the slow default AI reaches for — vectorise or `groupby` instead.

Handle the failure you can name

```
# First draft: hides the reason, and never closes the file
try:
    data = json.loads(open("config.json").read())
except:                                # also swallows Ctrl+C and your own typos
    data = {}

# What to ship
from pathlib import Path
log = logging.getLogger(__name__)
try:
    data = json.loads(Path("config.json").read_text(encoding="utf-8"))
except FileNotFoundError:
    log.warning("config.json missing; falling back to defaults")
    data = {}
except json.JSONDecodeError as e:
    raise ValueError(f"config.json is malformed: {e}") from e
```

Three questions per `try`: which exceptions can this actually raise, which do I want to recover from, and does the handle close if it blows up? Always pass `encoding`. And `print()` is not logging.

You supply the cases. AI writes the harness.

```
@pytest.mark.parametrize("gross,refund,expected", [
    (100.0, 0.0, 100.0),
    (100.0, 100.0, 0.0),      # fully refunded
    ( 0.0, 0.0, 0.0),        # empty order
    (100.0, 150.0, 0.0),     # refund > order -> clamp at 0
])
def test_net_amount(gross, refund, expected):
    assert net_amount(gross, refund) == pytest.approx(expected)

def test_missing_amount_is_rejected():
    with pytest.raises(ValueError):
        net_amount(None, 0.0)
```

- **The standing checklist:** empty, one, many · zero and negative · `None` · duplicates · non-ASCII names · timezone and month boundaries · input too big for memory.
- **Why you pick them:** a model optimises for tests that pass, not for the case that takes production down.

Three patterns to reject on sight

```
# 1. eval on anything a user can influence = code execution
rule = eval(user_input)           # never
rule = ast.literal_eval(user_input) # data only, no code

# 2. A hard-coded key is in git history forever
client = OpenAI(api_key="sk-proj-9f3a...") # never
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

# 3. shell=True turns a filename into a command
subprocess.run(f"convert {path}", shell=True) # never
subprocess.run(["convert", path])             # list args, no shell
```

Also review on sight: `pickle.load`, `yaml.load` without `SafeLoader`, `verify=False`, and f-strings built into SQL. **29 million** hard-coded secrets hit public GitHub in 2025, and AI-assisted commits leaked at roughly **double** the baseline rate. If one shipped: rotate it, don't just delete the line.

Pin it, and check the package is real

```
python -m venv .venv
.venv\Scripts\activate          # Windows    (source .venv/bin/activate)
pip install -r requirements.txt
pip freeze > requirements.lock  # exact versions, reproducible run

# requirements.txt
pandas==2.2.3                  # pinned: same build next month
pandas>=2                      # unpinned: a silent upgrade away
```

- **Verify every unfamiliar import** before installing: does it exist on PyPI, does the repo match, when was the last release, is the spelling exactly right?
- **Never** `pip install` **a name you first saw in a chat window.** Attackers now register hallucinated package names on purpose — the same fakes recur, so they are worth squatting.
- **One environment per project, never global installs** — and commit the lock file, so "works on my machine" stops being a defence.

Write it for the person reviewing it

```
def net_amount(gross: Decimal, refund: Decimal) -> Decimal:
    '''Revenue after refunds. Never returns a negative.'''
    if gross is None:
        raise ValueError("gross is required")
    return max(Decimal("0"), gross - refund)
```

Mutable default: one list shared by every call, forever

```
def add(item, bucket=[]):          # wrong
def add(item, bucket=None):        # right
    bucket = [] if bucket is None else bucket
```

```
0.1 + 0.2 == 0.3                  # False - never use float for money
```

Why it matters: across 211M changed lines, copy-pasted code rose from 8.3% to 12.3% while refactoring signals fell by more than half. Type hints, docstrings, small functions and a `ruff` + `mypy` pass are how volume stays reviewable.

Six checks before you merge it

- 1** I can explain every line in my own words.
- 2** Types and shapes verified on real data, not assumed.
- 3** Errors are specific and logged — no bare `except`, no swallowed failures.
- 4** Tests cover empty, zero, `None`, duplicate and too-big.
- 5** No secrets, no `eval`, no `shell=True` anywhere in the diff.
- 6** Dependencies pinned and every package confirmed to exist.

Save this — then run check #1 on the last Python file AI wrote for you.

Follow **Sagar Rathkanthiwar** for practical Data & AI

Sources: DORA, State of AI-assisted Software Development 2025 (dora.dev) · Veracode 2025 GenAI Code Security Report (veracode.com) · GitGuardian State of Secrets Sprawl 2026 (gitguardian.com) · Spracklen et al., package hallucination study, USENIX Security 2025 · GitClear AI Code Quality Research 2025 (gitclear.com). Examples are illustrative; verify behaviour against your Python and library versions.